System Design — Day 7

Topic: Key-Value Store (partitioning, consistent hashing, replication, quorum)

1. The Problem and Requirements

A key-value store holds pairs like **user_42 → {data}** and supports **put(key, value)** and **get(key)**. Examples: DynamoDB, Redis, Cassandra. Our case: 10 TB of data, machines hold about 1 TB each, and the store must survive machine failures. Questions to ask first:

- Reads and writes per second, and the read:write ratio?
- Typical value size (bytes, kilobytes, megabytes)?
- Consistency or availability if a machine is unreachable (CP or AP, Day 4)?
- Latency target, e.g. 99% of requests under 10 ms?
- Do keys expire (TTL)?

App servers stay stateless (Day 1), but the machines holding the data are stateful by nature, which is why losing one matters.

2. Splitting Data Across Machines

Why not "whichever machine is free"? The key must decide where the data lives. If a put lands on Machine 3 because it was free, a later get may ask Machine 8 and find nothing. The only fix would be asking every machine, which defeats the purpose of splitting the data.

Hashing the key: a hash function turns any key into a number, the same number every time. **machine = hash(key) mod N**. Both put and get run the same formula, so they always agree, with no searching. A good hash also spreads keys evenly.

3. The Problem with hash mod N

Adding a machine changes N, so the formula changes. Example: hash = 2,053. With 10 machines, 2,053 mod 10 = 3 (Machine 3). With 11 machines, 2,053 mod 11 = 7 (Machine 7). The data is still on Machine 3, so a get finds nothing.

About **10 out of every 11 keys (~91%)** change machines. For 10 TB, that is about **9.09 TB** to move while serving live traffic, and many lookups would miss in the meantime (the Day 3 cache stampede, applied to the whole data layer).

4. Consistent Hashing (the Ring)

Place machines on a circle by hashing their names. Hash each key onto the same circle, then **walk clockwise to the first machine**. That machine owns the key. Each machine owns the stretch of ring behind it, back to the previous machine. The ring wraps around, so after the highest position comes 0.

Worked example, ring positions 0-99, machines A=10, B=40, C=70, D=90:

Key position	Walk clockwise	Owner
25	25 → 40	B
75	75 → 90	D
95	95 → wraps to 0 → 10	A
40	already on B	B

- **Adding** machine E at 55: it takes positions 41-55 from C. Nobody else is touched.
- **Removing** B at 40: its keys fall to the next machine clockwise, C. A and D are untouched.
- **Fast lookup:** keep machine positions in a sorted list and binary-search for the first position at or above the key.

Going from 10 to 11 machines, the new machine takes about 1 in 11 keys: **~0.91 TB** of 10 TB, versus ~9.09 TB with mod N. That is a tenth of the movement, affecting only one neighbor.

5. Virtual Nodes

Problem with the basic ring: machine positions land randomly, so slices are uneven, and when a machine dies, its whole slice dumps onto one neighbor, which may overload and die too (a **cascading failure**).

Fix: place each machine many times (A#1, A#2, ... up to hundreds of spots). Each machine then owns many small slices scattered around the ring. Load evens out, a failed machine's slices spread across many neighbors, and a new machine takes a little from everyone. Stronger machines can be given more virtual nodes.

6. Replication on the Ring

Store each key on its owner and on the next 2 distinct machines clockwise: **N = 3 copies**. If a machine dies, the other copies answer the get.

- Storage: 10 TB × 3 copies = 30 TB, which is **30 machines** of 1 TB (plus headroom, since machines should not run 100% full).
- Lookups stay fast: a get goes to one machine (or a few) via the ring, it does not scan all copies.
- Writes get costlier: a put must reach 3 machines instead of 1.
- Real downside: copies can disagree (replication lag, Day 2; CAP trade-off, Day 4).

7. Quorum: N, W, R

- **N** = number of copies of each key.
- **W** = copies that must confirm a write before telling the user "saved".

- **R** = copies a read must ask; it returns the newest version (each value carries a version number).

Rule: if **R + W > N**, the read group and write group always overlap in at least one machine, so every read includes a copy with the latest write. Example with N=3, W=2, R=2: any 2 machines you read must include at least one of the 2 that took the write.

W	R	R + W	Result
1	1	2	Fastest reads and writes, but can return stale data. Favors availability (AP).
2	2	4	Balanced and the common choice. Fresh reads; survives one machine down.
3	1	4	Fresh reads and fast reads, but one dead machine blocks every write.
1	3	4	Fast writes and fresh reads, but one dead machine blocks every read.

Read repair: when a read notices a stale copy, it writes the newest value back to it, so stale copies heal over time. Conflicting simultaneous writes still need version tracking, which real systems add on top.

8. Key Takeaway

The key decides where its data lives (hashing). A ring keeps that mapping stable as machines come and go, virtual nodes even out the load, replication on the ring survives failures, and R + W > N lets you tune between speed and freshness. This is the CAP trade-off from Day 4 as an adjustable setting.